

PL/SQL MCQ Theory Synthesis

1. PL/SQL Fundamentals & Block Structure

Core Theory

A PL/SQL block has three sections: **DECLARE** (optional, for variables/cursors/exceptions), **BEGIN** (mandatory, executable statements), and **EXCEPTION** (optional, error handlers). The block ends with **END;**. A PL/SQL program is a **block-structured language** that supports anonymous blocks, named procedures, functions, packages, and triggers. The PL/SQL engine processes procedural statements; when it encounters an SQL statement, it passes it to the **SQL engine**.

Course Extension: The PL/SQL execution environment consists of the PL/SQL statement executor and the SQL statement executor. Switching between them incurs overhead called a **context switch**.

MCQ Hotspots

- `DBMS_OUTPUT.PUT_LINE` requires `SET SERVEROUTPUT ON` to display results.
- Anonymous blocks are not stored in the database; procedures/functions/packages/triggers are.
- **Context switch** occurs when the PL/SQL engine encounters an SQL statement and transfers it to the SQL engine. This is the performance cost of mixing SQL and PL/SQL.

Trap Map

- Distractor: "DBMS_OUTPUT works without SET SERVEROUTPUT ON" → FALSE. Output is suppressed by default.
 - Distractor: "Anonymous blocks are stored in the database" → FALSE. Only named subprograms are stored.
-

2. Variables, Constants & Data Types

Core Theory

Variables are declared with `[CONSTANT] datatype [NOT NULL] [:= | DEFAULT expr]`. Use `%TYPE` to anchor a variable to a table column or another variable. Use `%ROWTYPE` to declare a record matching an entire table row. **Substitution variables** (`&var`, `&&var`) are *SQLPlus constructs for user input*. ****Bind variables**** (`:var`, declared with `VARIABLE` in SQLPlus) persist across PL/SQL blocks and can be passed to subprograms.

Course Extension: PL/SQL introduces scalar types (`PLS_INTEGER`, `BINARY_INTEGER`, `BOOLEAN`) not available in SQL. Composite types include **RECORD** (`%ROWTYPE` or user-defined) and **collections** (index-by tables, nested tables, `VARRAYs`). `PLS_INTEGER` is faster than `NUMBER` for integer calculations.

`DECODE` vs `CASE`: `DECODE` is a SQL function (limited support in PL/SQL, works only inside SQL statements). `CASE` is a PL/SQL construct. `CASE` is preferred for procedural logic; `DECODE` is a compact SQL expression.

MCQ Hotspots

- `%TYPE` copies the **data type** but **not** constraints like `NOT NULL`.
- `BOOLEAN` variables can only hold `TRUE`, `FALSE`, or `NULL`. They cannot be used in SQL statements.
- Bind variables are declared in the **host environment** (SQL*Plus) and referenced with `:var_name` inside PL/SQL.

Trap Map

- Distractor: "%TYPE copies the NOT NULL constraint" → FALSE. Only the data type is copied.

- Distractor: "You can use BOOLEAN in SQL" → FALSE. BOOLEAN is PL/SQL only.
- Distractor: "Substitution variables persist after the block" → FALSE. They are preprocessor directives, not true variables.

3. Control Structures & Program Flow

Core Theory

PL/SQL provides three conditional structures: `IF-THEN-ELSE`, `ELSIF`, and `CASE` (both expression and statement forms). It provides three loop structures: `LOOP` (with `EXIT WHEN`), `WHILE`, and `FOR` (numeric and cursor). The `FOR` loop counter is **implicitly declared** and **read-only**.

Course Extension: The `NULL` statement is valid and explicitly says "no action needed." `GOTO` is rarely used and cannot jump into an `IF`, `CASE`, `LOOP`, or sub-block.

MCQ Hotspots

- `FOR` loop counter is **read-only**; modifying it inside the loop causes a compilation error.
- In a `FOR` loop, `EXIT WHEN` can terminate the loop early.
- `CASE` treats `NULL` differently than `IF` — `NULL = NULL` evaluates to `NULL` (FALSE in `IF`), so `CASE` can be safer for `NULL` handling.

Trap Map

- Distractor: "You can modify the `FOR` loop counter" → FALSE. It is read-only.
- Distractor: "`NULL = NULL` is TRUE in `IF`" → FALSE. It is `NULL`, which is treated as FALSE in `IF` conditions.

4. Cursors & Record Processing (★★★★ Highest MCQ Weight)

Core Theory

Implicit Cursors: Automatically created for every SQL statement in PL/SQL. Oracle manages them automatically. You reference them via `SQL%attribute`.

Explicit Cursors: Programmer-declared with `DECLARE CURSOR c IS SELECT . . . ;`. You control them with `OPEN`, `FETCH`, `CLOSE`. The `FOR` loop cursor (`FOR r IN c LOOP`) automatically handles `OPEN`, `FETCH`, and `CLOSE`.

Cursor Attributes:

- `%ISOPEN`: TRUE if cursor is open. For **implicit cursors**, this is **always FALSE**.
- `%FOUND`: TRUE if last `FETCH` succeeded; FALSE if not; `NULL` before first `FETCH`.
- `%NOTFOUND`: Opposite of `%FOUND`. TRUE if no row fetched; FALSE if row fetched; `NULL` before first `FETCH`.
- `%ROWCOUNT`: Number of rows fetched so far. For explicit cursors, it is **0** after `OPEN` and **before** the first `FETCH`.

Course Extension (Critical):

State	<code>%ISOPEN</code>	<code>%FOUND</code>	<code>%NOTFOUND</code>	<code>%ROWCOUNT</code>
Before <code>OPEN</code>	FALSE	-	-	-
After <code>OPEN</code> , before <code>FETCH</code>	TRUE	NULL	NULL	0
After successful <code>FETCH</code>	TRUE	TRUE	FALSE	N

After failed FETCH	TRUE	FALSE	TRUE	N
After CLOSE	FALSE	FALSE	TRUE	N

Implicit cursor special rules:

- `SQL%ISOPEN` is **always FALSE** (implicit cursors close automatically after execution).
- `SQL%ROWCOUNT`, `SQL%FOUND`, `SQL%NOTFOUND` are **NULL** if **no SQL statement was previously run** in the current block.
- After DML affecting 0 rows: `SQL%ROWCOUNT` = 0, `SQL%NOTFOUND` = TRUE, `SQL%FOUND` = FALSE.
- After `SELECT INTO` returning no rows: raises `NO_DATA_FOUND` (not caught by `SQL%NOTFOUND`).

Parameterized Cursors: Accept parameters to reuse the same query with different values: `CURSOR c(p_id NUMBER) IS SELECT ... WHERE col = p_id;` Open with `OPEN c(value);`

FOR UPDATE: Locks rows for the current transaction. `FOR UPDATE OF col1, col2 [NOWAIT | WAIT n]`. Use `WHERE CURRENT OF cursor_name` to update the current row without specifying a `WHERE` clause.

REF CURSOR / Dynamic Cursors: Cursor variables (`SYS_REFCURSOR`) can be opened for different queries at runtime. They are **weakly typed** (not bound to a specific row type). `OPEN c FOR 'SELECT ... WHERE col = :1' USING value;`

BULK COLLECT: Fetches multiple rows into a collection in a single operation, reducing context switches. `SELECT ... BULK COLLECT INTO collection FROM ....` Use `FETCH cursor BULK COLLECT INTO collection [LIMIT n]` to control memory usage.

FORALL: Executes multiple DML statements in a single context switch. `FORALL i IN collection.FIRST..collection.LAST UPDATE ... WHERE id = collection(i);` After `FORALL`, `SQL%ROWCOUNT` returns the **total number of rows affected** across all iterations.

`FORALL SAVE EXCEPTIONS`: If one DML statement in the `FORALL` fails, the entire `FORALL` stops. To continue processing, use `FORALL i IN collection.FIRST..collection.LAST SAVE EXCEPTIONS ....` After `FORALL`, `SQL%BULK_EXCEPTIONS(i).ERROR_INDEX` tells you which collection index caused the error, and `SQL%BULK_EXCEPTIONS(i).ERROR_CODE` gives the error code.

`SQL%BULK\_ROWCOUNT`: A composite attribute designed for `FORALL`. The *i*th element stores the number of rows processed by the *i*th execution of an `UPDATE` or `DELETE` statement. If the *i*th execution affects no rows, `SQL%BULK_ROWCOUNT(i)` returns zero.

`SQL%BULK\_EXCEPTIONS`: Stores information about exceptions encountered by a `FORALL` that uses `SAVE EXCEPTIONS`. A loop is required to determine where the exception occurred. `SQL%BULK_EXCEPTIONS(i).ERROR_INDEX` specifies which iteration of the `FORALL` loop caused the exception.

Nested Cursors: An inner parameterized cursor can be opened inside an outer cursor loop. This is useful for displaying hierarchical data (e.g., customers and their orders). The inner cursor is automatically closed after each iteration of the outer loop.

`%ROWCOUNT` for SELECT vs DML: `%ROWCOUNT` is not very useful for `SELECT` statements without `BULK COLLECT` in the case of implicit cursors. It is most useful for DML statements (`UPDATE`, `DELETE`, `INSERT`).

MCQ Hotspots

1. Implicit cursor ``SQL%ISOPEN`` is always **FALSE** — tested in MCQs #12, #15.

2. `%ROWCOUNT` is 0 after OPEN, before first FETCH — tested in MCQs #7, #15.
3. `%FOUND` and `%NOTFOUND` are NULL before first FETCH — tested in MCQs #7, #22.
4. You cannot manually OPEN/CLOSE a cursor used in a FOR loop — doing so causes a compilation error (MCQ #8).
5. `FOR UPDATE` with `WAIT n` times out and throws an exception (ORA-30006), not `NO_DATA_FOUND` — tested in MCQ #17.
6. `FORALL` is recommended to reduce context switches — tested in MCQ #5.
7. After `FORALL`, `SQL%ROWCOUNT` shows total rows affected — tested in MCQ #4.

Trap Map

- **Distractor:** "SQL%ROWCOUNT is 0 before any SQL" → FALSE. It is **NULL** before the first SQL statement.
- **Distractor:** "SQL%ISOPEN is TRUE after an UPDATE" → FALSE. It is **always FALSE** for implicit cursors.
- **Distractor:** "c%ROWCOUNT is 1 after OPEN" → FALSE. It is **0** after OPEN.
- **Distractor:** "c%FOUND is FALSE after OPEN" → FALSE. It is **NULL** before the first FETCH.
- **Distractor:** "FOR UPDATE timeout raises NO_DATA_FOUND" → FALSE. It raises a **resource busy** exception (ORA-30006).
- **Distractor:** "FORALL is for readability" → FALSE. It is for **reducing context switches**.
- **Distractor:** "WHERE CURRENT OF works without FOR UPDATE" → FALSE. FOR UPDATE is mandatory.

5. Error Handling & Exception Management

Core Theory

PL/SQL has three exception categories:

1. **Predefined:** Oracle-named exceptions (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, INVALID_CURSOR, VALUE_ERROR, INVALID_NUMBER, DUP_VAL_ON_INDEX). Raised automatically.
2. **Non-predefined:** Oracle errors without names. Use PRAGMA EXCEPTION_INIT(exception_name, -error_code) to name them.
3. **User-defined:** Programmer-declared. Must be raised explicitly with RAISE.

Exception Propagation: If an exception is not handled in the current block, it propagates to the enclosing block. If unhandled at the top level, the program terminates.

Error Code Retrieval:

- `SQLCODE`: Returns the error code. Returns +100 for NO_DATA_FOUND, negative for Oracle errors, +1 for user-defined exceptions without PRAGMA, and 0 for no error.
- `SQLERRM`: Returns the error message. `SQLERRM(0)` returns ORA-0000: normal, successful completion.
- A SQL statement cannot invoke `SQLCODE` or `SQLERRM` directly.

RAISE_APPLICATION_ERROR: Creates user-defined error messages. Error number must be between -20000 and -20999. The optional third parameter (TRUE/FALSE) controls whether the error is added to the stack or replaces current errors.

Nested Exception Handling: Inner blocks can have their own handlers. If an exception is handled in an inner block, it does not propagate. If re-raised with RAISE, it propagates.

MCQ Hotspots

- NO_DATA_FOUND has error code **+100**, not negative.

- `SQLCODE` and `SQLERRM` are **only meaningful inside exception handlers**. Outside them, they return 0 and `ORA-0000`.
- `WHEN OTHERS` should be the **last handler** in the `EXCEPTION` section.
- `RAISE_APPLICATION_ERROR` only works inside **executing stored subprograms** (or methods), not in anonymous blocks.

Trap Map

- Distractor: "`NO_DATA_FOUND` has a negative error code" → FALSE. It is **+100**.
- Distractor: "You can use `SQLCODE` in a `SELECT` statement" → FALSE. A SQL statement cannot invoke `SQLCODE` or `SQLERRM`.
- Distractor: "`RAISE_APPLICATION_ERROR` works in anonymous blocks" → FALSE. It only works in stored subprograms.
- Distractor: "`WHEN OTHERS` should be first" → FALSE. It should be **last**.

6. Stored Procedures & Functions

Core Theory

Procedures: Named PL/SQL blocks that perform actions. Can return values via `OUT` or `IN OUT` parameters. Created with `CREATE [OR REPLACE] PROCEDURE`.

Functions: Named PL/SQL blocks that **must return a value**. Created with `CREATE [OR REPLACE] FUNCTION ... RETURN datatype`. To be used in **SQL statements**, a function must:

- Have only **IN** parameters
- Return a valid SQL data type
- Not contain `COMMIT` or `ROLLBACK`
- Not contain DML if used in `SELECT` (no `INSERT/UPDATE/DELETE` on the same table)
- Not contain DDL or DCL (`EXECUTE IMMEDIATE`)

Parameter Modes:

- `IN`: Passes value in (default). Read-only.
- `OUT`: Returns value. **Uninitialized** when the procedure starts.
- `IN OUT`: Both. Must have a value to pass in, can be modified.

Calling Conventions:

- `EXEC procedure_name(params)` — SQL*Plus command, wraps in anonymous block.
- `CALL procedure_name(params)` — SQL statement, can inhibit exception propagation.
- Named notation: `proc(p1=>val1, p2=>val2)`.
- Mixed notation: `proc(val1, p2=>val2)` — OK. `proc(p1=>val1, val2)` — NOT OK.

Course Extension (Critical):

- **AUTHID:** `AUTHID DEFINER` (default) runs with owner's privileges. `AUTHID CURRENT_USER` runs with caller's privileges.
- **AUTONOMOUS_TRANSACTION:** A subprogram with `PRAGMA AUTONOMOUS_TRANSACTION` runs in a separate transaction. It must `COMMIT` or `ROLLBACK` before exiting. Changes are visible immediately, but the parent transaction cannot see them until it commits.

- **RESULT_CACHE:** For recursive functions, `RESULT_CACHE` stores function results so repeated calls with the same parameters return the cached value without re-executing. This dramatically improves performance for expensive recursive calculations (e.g., Fibonacci).
- **DETERMINISTIC:** `PRAGMA DETERMINISTIC` tells Oracle that a function returns the same result for the same inputs. This allows the optimizer to cache results and avoid redundant calls. It is required for functions used in function-based indexes and materialized views.

MCQ Hotspots

- Functions returning `BOOLEAN` **cannot be used in SQL statements**.
- Functions used in SQL **cannot have OUT or IN OUT parameters**.
- OUT parameters are **uninitialized** when the procedure starts; they must be assigned a value.
- `EXEC` uses less memory than `CALL`; `CALL` is a SQL statement and can inhibit exception propagation.
- `AUTHID DEFINER` is the default; `AUTHID CURRENT_USER` runs with the caller's privileges.
- `AUTONOMOUS_TRANSACTION` requires `COMMIT/ROLLBACK` inside the procedure.

Trap Map

- Distractor: "A function with OUT parameters can be used in SELECT" → FALSE. Functions in SQL must have only **IN** parameters.
- Distractor: "OUT parameters are initialized to the input value" → FALSE. They are **uninitialized**.
- Distractor: "CALL is the same as EXEC" → FALSE. `CALL` can inhibit exception propagation.
- Distractor: "AUTHID CURRENT_USER is the default" → FALSE. The default is `AUTHID DEFINER`.

7. Advanced PL/SQL Features (★★★ High MCQ Weight)

7.1 Triggers

Core Theory

A trigger is a PL/SQL block that executes automatically when a specified event occurs. Components:

- **Timing:** `BEFORE`, `AFTER`, or `INSTEAD OF` (for views)
- **Event:** `INSERT`, `UPDATE`, `DELETE` (can combine with `OR`)
- **Level:** `FOR EACH ROW` (row-level) or statement-level (default)
- **Condition:** `WHEN (condition)` — optional

Pseudo-records in row-level triggers:

- `:OLD` — value before the operation
- `:NEW` — value after the operation
- For `INSERT`: `:OLD` is **NULL**
- For `DELETE`: `:NEW` is **NULL**
- For `UPDATE`: both have values

Operation Detection:

- `IF INSERTING THEN ... END IF;`
- `IF UPDATING THEN ... END IF;`
- `IF DELETING THEN ... END IF;`
- These work in **any** trigger (row or statement level).

Statement-level trigger: Without `FOR EACH ROW`, the trigger fires **once per DML statement**, regardless of how many rows are affected. Even if 0 rows are affected, it still fires.

INSTEAD OF Triggers: Used for views that are not otherwise updatable. When you execute `INSERT`, `UPDATE`, or `DELETE` on a view, the `INSTEAD OF` trigger fires and you write custom logic to modify the underlying base tables. `INSTEAD OF` triggers are **always row-level** (they automatically work per row, even without `FOR EACH ROW`).

MCQ Hotspots

- A trigger **without** `FOR EACH ROW` fires **once per statement**, not per row — tested in MCQ #6.
- `INSERTING/UPDATING/DELETING` tests work in **any** trigger (row or statement) — tested in MCQ #11.
- `:OLD` is `NULL` for `INSERT`; `:NEW` is `NULL` for `DELETE`.
- Statement-level triggers fire even if 0 rows are affected.

Trap Map

- Distractor: "A statement-level trigger fires only if at least one row is affected" → FALSE. It fires **regardless** of row count.
 - Distractor: "`INSERTING` only works in `BEFORE` triggers" → FALSE. It works in **any** trigger.
 - Distractor: "`:OLD` has a value for `INSERT`" → FALSE. `:OLD` is **NULL** for `INSERT`.
-

7.2 Packages

Core Theory

A package groups related subprograms, variables, and types. It has two parts:

- **Specification:** Declares the public interface (procedures, functions, variables, types).
- **Body:** Contains the implementation.

Package variables (declared in the specification) persist for the **duration of the current session** and can store state between calls to any subprogram in the package.

Course Extension:

- If the specification is missing, the **body will not compile**.
- A package can have a **specification without a body** (if it only contains constants, types, or variables).
- A package **cannot have a body without a specification**.
- The body can be recompiled independently if the specification hasn't changed.

MCQ Hotspots

- Package variables persist for the **session**, not all sessions simultaneously.
- Without a specification, the body **will not compile** — tested in MCQ #10.
- A spec-only package is valid; a body-only package is **not**.

Trap Map

- Distractor: "Package variables are shared across all sessions" → FALSE. They are **per-session**.
 - Distractor: "A package body can exist without a specification" → FALSE. The spec is mandatory.
 - Distractor: "Package variables are destroyed at `COMMIT`" → FALSE. They persist for the **session**.
-

7.3 Collections (VARRAY, Nested Table, Associative Array)

Core Theory

Associative Array (Index-by Table):

- Indexed by arbitrary keys (VARCHAR2 or PLS_INTEGER)
- Exists only in memory (PL/SQL only)
- Can be sparse (non-sequential keys)
- Methods: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT
- **No `LIMIT` method** (no maximum size)

Nested Table:

- Stored in database (can be a column type)
- Initially dense, but can become sparse after DELETE (n)
- Methods: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT, EXTEND, TRIM, DELETE
- Stored in a separate storage table

VARRAY (Variable-size Array):

- Has a **maximum size** declared at creation
- Dense (always has consecutive subscripts)
- Methods: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT, EXTEND, TRIM, LIMIT, DELETE
- DELETE only works for **all elements** (cannot delete a single element)
- DELETE (n) is invalid for VARRAYs
- Index starts at 1, fixed lower bound

Collection Comparison Table (Course Critical):

Feature	Associative Array	Nested Table	VARRAY
Declared with	INDEX BY key_type	TABLE OF element_type	VARRAY(max_size) OF element_type
Storage	Memory only (PL/SQL)	Database (column type)	Database (column type)
Index	Arbitrary (VARCHAR2, PLS_INTEGER)	Sequential 1..N	Sequential 1..max_size
Can be sparse	Yes	Yes (after DELETE)	No (always dense)
Max size	Unlimited (no LIMIT)	No LIMIT	Fixed at declaration
Stored in DB	No	Yes (separate storage table)	Yes
EXISTS	Yes	Yes	Yes
COUNT	Yes	Yes	Yes
FIRST/LAST	Yes	Yes	Yes
PRIOR/NEXT	Yes	Yes	Yes
LIMIT	No	Yes	Yes
EXTEND	No	Yes	Yes
TRIM	No	Yes	Yes
DELETE (n)	Yes	Yes	No (only DELETE all)

Can be NULL	No (elements can be)	Yes (entire collection)	Yes (entire collection)

Collection Assignment Rules:

- You can assign one collection to another only if they are the **same type** (declared with the same TYPE statement or same schema-level type).
- `t1 := t2` works if both are the same type.
- You cannot assign an associative array to a nested table or VARRAY.
- A collection can be NULL (not just empty). You must initialize it with a constructor before use.

Collection Methods (All Types):

- EXISTS (n) : Returns TRUE if element at index n exists
- COUNT: Returns number of elements
- FIRST/LAST: Returns the **first and last defined index** (not the values, not the count)
- PRIOR (n) /NEXT (n) : Returns the index before/after n
- LIMIT: Returns maximum size (VARRAY only)
- EXTEND/EXTEND (n) /EXTEND (n, i) : Adds elements (nested tables only)
- TRIM/TRIM (n) : Removes elements from the end (nested tables and VARRAYs)
- DELETE/DELETE (n) /DELETE (n, m) : Removes elements (not for VARRAYs when n is specified)

MCQ Hotspots

- VARRAY** has a **fixed maximum size** — tested in MCQ #9.
- DELETE (n) **cannot** remove a single element from a VARRAY — tested in MCQ #9.
- FIRST and LAST return **index values**, not element values or count — tested in MCQ #14.
- Nested tables can become **sparse** after DELETE — tested in MCQ #16.
- LIMIT only works for **VARRAYs and nested tables** (not associative arrays).
- VARRAYs are **always dense** (no gaps after DELETE of single element).

Trap Map

- Distractor: "VARRAY is always sparse" → FALSE. It is **always dense**.
- Distractor: "DELETE (n) works on VARRAY" → FALSE. It only works for **all elements**.
- Distractor: "FIRST and LAST return the values" → FALSE. They return **index values**.
- Distractor: "Associative arrays have a LIMIT method" → FALSE. Only VARRAYs and nested tables have LIMIT.
- Distractor: "VARRAY index can be negative" → FALSE. Index starts at **1**.

7.4 Dynamic SQL

Core Theory

EXECUTE IMMEDIATE: Executes SQL statements constructed at runtime. Supports INTO for single-row results and USING for bind variables. Can execute DDL inside PL/SQL (normally impossible). Causes **hard parse** each time unless bind variables are used.

REF CURSOR: A cursor variable that can be associated with different queries at runtime. Two types:

- Strong:** TYPE rc IS REF CURSOR RETURN table%ROWTYPE; — bound to a specific row type.
- Weak:** SYS_REFCURSOR — not bound to any row type.

Course Extension:

- `OPEN cursor FOR dynamic_string USING bind_vars;` — used with REF CURSOR.
- You cannot use a `FOR r IN cursor_variable LOOP` syntax with a REF CURSOR variable. You must use `FETCH` in a loop.

MCQ Hotspots

- `EXECUTE IMMEDIATE` supports `INTO` for single-row results and `USING` for bind variables.
- `SYS_REFCURSOR` is a **weak** REF CURSOR.
- You cannot use a `FOR` loop directly with a `SYS_REFCURSOR` variable; you must use `FETCH`.

Trap Map

- Distractor: "`FOR r IN sys_refcursor LOOP` works" → FALSE. Use `FETCH` instead.
- Distractor: "`EXECUTE IMMEDIATE` is the only way to run DDL in PL/SQL" → FALSE. `DBMS_SQL` is another option.

Summary: The 12 Oracle Gotchas That Become MCQs

1. `SQL%ISOPEN` is **always FALSE** for implicit cursors.
2. `SQL%ROWCOUNT` is **NULL** before any SQL statement, **0** after DML affecting 0 rows.
3. `c%ROWCOUNT` is **0** after `OPEN`, before first `FETCH`.
4. `c%FOUND` and `c%NOTFOUND` are **NULL** before first `FETCH`.
5. **FOR loop cursors** auto-handle `OPEN/FETCH/CLOSE` — you cannot manually do it.
6. `FORALL` reduces **context switches** — that's its primary purpose.
7. A **statement-level trigger** (no `FOR EACH ROW`) fires **once per statement**, not per row.
8. `INSERTING/UPDATING/DELETING` works in **any** trigger (row or statement).
9. A **package body will not compile without a specification**.
10. A **VARRAY** has a **fixed maximum size** and is **always dense**.
11. `DELETE (n)` **does not work** on a VARRAY (only works for all elements).
12. `FIRST/LAST` return **index values**, not element values or count.

How to Read a PL/SQL Code Block for MCQs

1. **Check for implicit vs explicit cursors.** If it's a `SELECT INTO`, `UPDATE`, `INSERT`, or `DELETE`, it's an implicit cursor. `SQL%ISOPEN` is always FALSE. `SQL%ROWCOUNT` is NULL before the statement, then shows the affected row count.
2. **Check cursor state.** If you see `OPEN c`, immediately note: `%ROWCOUNT` is 0, `%FOUND/%NOTFOUND` are NULL. After `FETCH`, check if it was successful.
3. **Check for FOR loop cursors.** If you see `FOR r IN c LOOP`, you cannot `OPEN` or `CLOSE c` manually. If the code tries, it's a compilation error.
4. **Check trigger level.** Look for `FOR EACH ROW`. If missing, it's statement-level (fires once per statement).
5. **Check collection type.** Look for `VARRAY`, `TABLE OF`, `INDEX BY`. Note the methods available and the density rules.
6. **Check exceptions.** If `NO_DATA_FOUND` is possible, note that it's a predefined exception with code +100. If `WHEN OTHERS` exists, it must be last.

Appendix: Course-Only Deep Dives

These topics are taught primarily in the courses (not emphasized in seminars) but appear on MCQs.

A.1 DECODE vs CASE

- DECODE is a SQL function: `DECODE(expr, val1, result1, val2, result2, default)`.
- CASE is a PL/SQL construct: `CASE WHEN condition THEN result ... END`.
- DECODE can only be used inside SQL statements within PL/SQL, not as procedural logic.
- CASE is more flexible and can handle complex conditions; DECODE only checks equality.

A.2 FORALL SAVE EXCEPTIONS & Bulk Error Handling

- Without `SAVE EXCEPTIONS`, one failure in `FORALL` stops the entire batch.
- With `SAVE EXCEPTIONS`, the `FORALL` continues and stores all errors in `SQL%BULK_EXCEPTIONS`.
- After `FORALL SAVE EXCEPTIONS`, you must loop through `SQL%BULK_EXCEPTIONS.COUNT` to find failures.
- `SQL%BULK_EXCEPTIONS(i).ERROR_INDEX` = the index in the collection that failed.
- `SQL%BULK_EXCEPTIONS(i).ERROR_CODE` = the Oracle error code.

A.3 SQL%BULK_ROWCOUNT

- Only meaningful with `FORALL`.
- `SQL%BULK_ROWCOUNT(i)` tells how many rows the *i*th DML statement affected.
- If the *i*th statement affects 0 rows, `SQL%BULK_ROWCOUNT(i)` returns 0.
- This is different from `SQL%ROWCOUNT` which returns the **total** after `FORALL`.

A.4 FETCH BULK COLLECT INTO ... LIMIT n

- Used with explicit cursors to avoid memory exhaustion.
- `FETCH c BULK COLLECT INTO collection LIMIT 100`; fetches at most 100 rows per call.
- You typically call this in a loop until `c%NOTFOUND`.
- This is the recommended pattern for large result sets.

A.5 Nested Cursors (Course Pattern)

```
DECLARE
  CURSOR c1 IS SELECT * FROM customers;
  CURSOR c2(cust_id NUMBER) IS SELECT order_id FROM orders WHERE customer_id = cust_id;
BEGIN
  FOR r1 IN c1 LOOP
    FOR r2 IN c2(r1.customer_id) LOOP
      -- inner cursor is automatically closed after each iteration
    END LOOP;
  END LOOP;
END;
```

- Inner cursor is automatically closed after each outer loop iteration.
- Parameter values are passed from the outer cursor to the inner cursor.

A.6 INSTEAD OF Triggers

- Only for views.
- Automatically row-level (no `FOR EACH ROW` needed).
- You write `UPDATE table1 ... WHERE id = :OLD.id` inside the trigger to modify base tables.
- `:OLD` and `:NEW` are available.

A.7 PRAGMA DETERMINISTIC

- Tells Oracle the function always returns the same result for the same input.
- Required for function-based indexes and materialized views.
- Allows Oracle to cache the result and avoid redundant calls.

- Example: `CREATE FUNCTION my_func(...) RETURN NUMBER DETERMINISTIC IS ...`

A.8 RESULT_CACHE for Recursive Functions

- `CREATE FUNCTION fibonacci(n NUMBER) RETURN NUMBER RESULT_CACHE IS ...`
- Stores results in a system cache. If called again with the same parameter, Oracle returns the cached result instead of re-executing.
- Dramatically improves performance for expensive recursive functions.

A.9 Collection Assignment & NULL

- A collection variable can be `NULL` (uninitialized) or empty (initialized but no elements).
- `t := NULL` is valid for nested tables and VARRAYs.
- `t IS NULL` checks if the collection is null; `t.COUNT = 0` checks if it is empty.
- `t(1)` raises `ORA-06531` if `t` is null, not if it is empty.

A.10 %ROWCOUNT for SELECT vs DML

- `%ROWCOUNT` is most useful for DML statements (`UPDATE`, `DELETE`, `INSERT`).
- For `SELECT` statements without `BULK COLLECT`, `%ROWCOUNT` is **not very useful** for implicit cursors.
- For explicit cursors, `%ROWCOUNT` increments after each successful `FETCH`.
- After `BULK COLLECT`, `%ROWCOUNT` reflects the total rows fetched.

A.11 Context Switching Deep Dive

- Every time PL/SQL encounters an SQL statement, the PL/SQL engine stops and passes the statement to the SQL engine.
- The SQL engine executes it and returns control.
- This transfer is called a **context switch** and adds overhead.
- `FORALL` and `BULK COLLECT` reduce context switches by batching operations.
- **Rule of thumb:** Row-by-row processing = many context switches = slow. Bulk processing = few context switches = fast.

A.12 AUTHID and AUTONOMOUS_TRANSACTION Deep Dive

- `AUTHID DEFINER` (default): The procedure runs with the privileges of the **owner**. The caller only needs `EXECUTE` privilege.
- `AUTHID CURRENT_USER`: The procedure runs with the privileges of the **caller**. The caller must have privileges on all underlying objects.
- `AUTONOMOUS_TRANSACTION`: The subprogram runs in a separate transaction. It can `COMMIT` or `ROLLBACK` independently.
- If you call an autonomous procedure and it commits, the changes are visible to other sessions immediately, but the parent transaction still sees the old data until it commits.
- If you don't `COMMIT`/`ROLLBACK` inside an autonomous transaction, Oracle raises an error when the procedure exits.

A.13 Additional Edge Cases & Course Details

These topics are present in the course materials but were not explicitly tested in the 22 MCQs. They may appear on future or similar exams.

A.13.1 `BINARY\_FLOAT` and `BINARY\_DOUBLE`

- Represent floating-point numbers in IEEE 754 format.
- Offer better interoperability and operational speed than `NUMBER`.

- NUMBER stores up to 38 digits; BINARY_FLOAT and BINARY_DOUBLE have lower precision but better performance for scientific calculations.
- BINARY_FLOAT: 35 digits; BINARY_DOUBLE: 33 digits (from course).

A.13.2 `SET AUTOPRINT ON` for Bind Variables

- SET AUTOPRINT ON automatically displays bind variable values after a PL/SQL block completes, without needing PRINT.
- If off, you must explicitly PRINT var_name to see the value.

A.13.3 Alternative String Literal Delimiters ('q'!'...'')

- q'!Father's day!' is a valid alternative to escaping single quotes inside strings.
- You can use any delimiter character: q'[...]', q'(...)', q'{...}', q'|...|', etc.
- This avoids the need to double single quotes ('Father' 's day').

A.13.4 `MINUS` Set Operator with `BULK COLLECT`

- SELECT * BULK COLLECT INTO t FROM employees MINUS SELECT * FROM employees WHERE ROWNUM <= 50; is valid.
- MINUS returns rows from the first query that are not in the second query.
- INTERSECT and UNION also work in this context.

A.13.5 `TABLE()` Function for Querying Nested Tables

- SELECT * FROM TABLE(SELECT degrees FROM location_degrees WHERE id_location = 1); returns individual elements of a nested table.
- SELECT id_location, name, b.* FROM location_degrees a, TABLE(a.degrees) b; returns the parent row plus each nested table element as a separate row.
- TABLE() is a SQL function that converts a collection into a relational result set.

A.13.6 `NESTED TABLE ... STORE AS ...` Storage Clause

- CREATE TABLE location_degrees (id_location NUMBER(4), name VARCHAR2(40), degrees nt_degrees) NESTED TABLE degrees STORE AS nested_degrees;
- The STORE AS clause creates a separate storage table for the nested table data.
- You can query the storage table directly using TABLE() function.

A.13.7 `GREATEST` vs `MAX` Behavior

- GREATEST(b, c) returns the larger of two values (works with any number of arguments).
- MAX(salary) is an aggregate function that returns the maximum value from a column.
- GREATEST is a scalar function; MAX is an aggregate function. You cannot use MAX in a simple assignment without a SELECT statement.

A.13.8 `ROWNUM` Filtering in Cursor Queries

- WHERE ROWNUM <= 50 limits the result set to 50 rows.
- ROWNUM is assigned before sorting, so ORDER BY must use a subquery if you need the top N rows in a specific order.
- Correct pattern: SELECT * FROM (SELECT * FROM employees ORDER BY salary DESC) WHERE ROWNUM <= 50.

A.13.9 `GROUP BY` / `HAVING` in Cursor Definitions

- HAVING filters groups after aggregation; WHERE filters rows before aggregation.
- SELECT department_id, AVG(salary) FROM employees GROUP BY department_id HAVING AVG(salary) > 5000; is valid in a cursor definition.

- HAVING can only reference columns that are in the GROUP BY clause or are aggregate functions.

A.13.10 `JOIN` Syntax in Cursor Queries

- `SELECT d.department_name, AVG(e.salary) FROM departments d JOIN employees e ON d.department_id = e.department_id GROUP BY d.department_name;` is valid in a cursor.
- `FROM table1, table2 WHERE table1.id = table2.id` (old join syntax) and `FROM table1 JOIN table2 ON table1.id = table2.id` (ANSI join syntax) both work.
- If using `FOR UPDATE`, use `FOR UPDATE OF table1.column` to specify which table's rows to lock.

Good luck on your exam!